Math Behind Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are designed to process data that has a known grid-like topology, such as images (which can be seen as 2D grids of pixels). The key components of a CNN include convolutional layers, pooling layers, activation functions, and fully connected layers. Each of these components relies on specific mathematical operations that allow the network to learn and extract features from input data.

Convolution Operation

The convolution operation is central to CNNs and involves sliding a filter (or kernel) across the input data to produce a feature map.

Convolution in 2D

For a 2D input image I and a 2D kernel K, the convolution operation can be defined as:

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i+m, j+n) \cdot K(m,n)$$

- $I(i,j)$: The pixel value at position $I(i,j)$ in the input image.
- $K(m,n)$: The weight of the kernel at position (m,n).
- $S(i,j)$: The output feature map at position (i,j).

Here's a more detailed breakdown:

- **Kernel:** A small matrix (e.g., 3×3 or 5×5) that slides over the image. Each position on the kernel has a weight $K(m,n)$.
- **Sliding Window:** The kernel is applied to each overlapping region of the image. For each position, we multiply the corresponding pixel values of the image and the kernel weights and then sum these products to get a single number. This process creates a new matrix, called the feature map, which highlights certain features of the image.

Example Calculation

Consider a simple 3×3 kernel applied to a 5×5 image. Suppose the image I and kernel K are:

$$I = \begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 0 & 1 & 2 & 3 & 0 \\ 1 & 0 & 1 & 2 & 3 \\ 3 & 2 & 1 & 0 & 1 \\ 0 & 1 & 0 & 3 & 2 \end{bmatrix}, K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

When the kernel is applied to the top-left 3×3 region of the image:

$$\text{Region} = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix}$$

The convolution sum $S(1,1)$ is:

$$\begin{aligned} S(1,1) &= (1 \times 1) + (2 \times 0) + (3 \times -1) + (0 \times 1) + (1 \times 0) + (2 \times -1) \\ &\quad + (1 \times 1) + (0 \times 0) + (1 \times -1) S(1,1) \\ &= 1 + 0 - 3 + 0 + 0 - 2 + 1 + 0 - 1 = -4 \end{aligned}$$

This value is placed in the output feature map at position (1, 1).

Padding and Stride in Convolutional Neural Network

Padding in CNNs

To preserve the spatial dimensions of the input, padding is applied. Padding adds extra rows and columns around the border of the input image.

- **Zero Padding:** The simplest form of padding, where additional pixels with a value of zero are added around the image.

If we apply zero padding of 1 pixel to our 5×5 image, it becomes 7×7:

$$I_{\text{padded}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 0 & 1 & 0 \\ 0 & 0 & 1 & 2 & 3 & 0 & 0 \\ 0 & 1 & 0 & 1 & 2 & 3 & 0 \\ 0 & 3 & 2 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 3 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

This ensures that when the kernel is applied, the output feature map remains the same size as the original input.

Stride

Stride controls how much the kernel shifts as it slides over the input image. For a stride of 1, the kernel moves one pixel at a time. For a stride of 2, it moves two pixels at a time, effectively down sampling the image.

The output size after applying a convolution operation with a given stride s and padding p can be calculated as:

$$\text{Output size} = \left\lfloor \frac{\text{Input size} - \text{Kernel size} + 2 \times p}{s} \right\rfloor + 1$$

For example, with an input size of 5, kernel size of 3, padding of 1, and stride of 1:

$$\text{Output size} = \left\lfloor \frac{5 - 3 + 2 \times 1}{1} \right\rfloor + 1 = \left\lfloor \frac{5}{1} \right\rfloor + 1 = 5$$

The output feature map will have the same spatial dimensions as the input.

Pooling Layers in Convolutional Neural Network

Pooling is a down-sampling operation used to reduce the dimensions of the feature maps while retaining the most important information. Max pooling is the most common pooling operation, where the maximum value within a window is selected.

Max Pooling:

For a 2×2 max pooling operation, consider the following example:

$$\text{Input Feature Map} = \begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 0 \\ 2 & 7 & 3 & 8 \\ 4 & 1 & 0 & 2 \end{bmatrix}$$

After applying a 2×2 max pooling with a stride of 2:

$$\text{Pooled Feature Map} = \begin{bmatrix} 6 & 4 \\ 7 & 8 \end{bmatrix}$$

Each value in the pooled feature map represents the maximum value within the corresponding 2×2 window in the input feature map.

Activation Functions

Activation functions introduce non-linearity into the network, allowing it to learn more complex representations.

ReLU (Rectified Linear Unit):

ReLU is a piecewise linear function defined as:

$$\text{ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$

ReLU simply replaces all negative values in the input with zero. This operation can be thought of as "activating" only those neurons that contribute to the network's decision-making.

Example:

For an input vector:

$$\mathbf{x} = \begin{bmatrix} -2 \\ 0.5 \\ 3 \\ -1.5 \end{bmatrix}$$

Applying ReLU gives:

$$\text{ReLU}(\mathbf{x}) = \begin{bmatrix} 0 \\ 0.5 \\ 3 \\ 0 \end{bmatrix}$$

Fully Connected Layers

Fully connected layers are where each neuron is connected to every neuron in the previous layer. Mathematically, this is a linear transformation followed by an activation function.

Matrix Multiplication in Fully Connected Layers

Given an input vector $\mathbf{x}$ and a weight matrix $\mathbf{W}$, the output $\mathbf{z}$ is calculated as:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

Where:

- $\mathbf{W}$ is the weight matrix.
- $\mathbf{b}$ is the bias vector.

- $\mathbf{z}$ is the output vector.

Example:

Let $\mathbf{x}$ be a 3-dimensional input vector, $\mathbf{W}$ a 2×3 weight matrix, and $\mathbf{b}$ a 2-dimensional bias vector:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \mathbf{W} = \begin{bmatrix} 0.2 & 0.8 & -0.5 \\ -0.3 & 0.4 & 0.9 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

Then, the output $\mathbf{z}$ is:

$$\mathbf{z} = \begin{bmatrix} 0.2 & 0.8 & -0.5 \\ -0.3 & 0.4 & 0.9 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$\mathbf{z} = \begin{bmatrix} (0.2 \times 1) + (0.8 \times 2) + (-0.5 \times 3) \\ (-0.3 \times 1) + (0.4 \times 2) + (0.9 \times 3) \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$\mathbf{z} = \begin{bmatrix} 0.2 + 1.6 - 1.5 \\ -0.3 + 0.8 + 2.7 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.3 \\ 3.0 \end{bmatrix}$$

Backpropagation and Gradient Descent in CNN

Backpropagation

Backpropagation is the process of calculating the gradient of the loss function with respect to each weight by the chain rule, allowing for efficient weight updates. This involves calculating the error at each layer and then propagating this error backward through the network.

Gradient Descent

Once the gradients are calculated, gradient descent is used to update the weights to minimize the loss function:

$$\theta := \theta - \alpha \cdot \nabla_\theta J(\theta)$$

Where:

- θ are the model parameters (weights and biases).
- α is the learning rate.
- $\nabla_\theta J(\theta)$ is the gradient of the loss function J with respect to θ.

Conclusion

The mathematics behind CNNs is crucial for understanding how they function and why they are effective in tasks like image recognition. By exploring concepts such as convolution, padding, stride, pooling, and backpropagation, we gain insight into the powerful capabilities of CNNs to learn and generalize from data. With this mathematical understanding, one can design, optimize, and apply CNNs to a wide range of real-world problems.